# VAD/CNG Software User's Guide

**Adaptive Digital Technologies, Inc.**
**November 05, 2015**
**Issue 5**

Copyright © 2000 - 2024, Adaptive Digital Technologies Inc.

**Revision History**

| Issue | Revision History | Date | Initials |
|---|---|---|---|
| 1 | Version 5.00 – restart | 1/27/2012 | SDK |
| 2 | Version 5.01 – Added arm, i686 support | 4/11/2012 | BY |
| 3 | Version 5.02 – Added CNG FIR length parameter | 6/21/2013 | RMF |
| 4 | Version 5.02 – Documented new API | 8/23/2013 | RMF |
| 5 | Fixed typo for the cng API | 11/05/2015 | HFZ |

## Table of Contents

**1. INTRODUCTION ..... 1**

**2. SOFTWARE API ..... 1**

2.1. CHANNEL INSTANTIATION ..... 1
2.2. VADCNG_ADT_INIT( ) ..... 1
2.3. VADCNG_ADT_VAD( ) ..... 2
2.4. VADCNG_ADT_CNG( ) ..... 2
2.5. VAD_ADT_GETSTATUS( ) ..... 3
2.6. VAD_ADT_GETINSTANCESIZE( ) ..... 3
2.7. VAD_ADT_LIBVERSION( ) ..... 4
2.8. VAD_ADT_APIVERSION( ) ..... 4

**3. SAMPLE HOST SOFTWARE ..... 4**

# 1. Introduction

The Adaptive Digital Technologies (ADT) VAD/CNG allows applications to run the ADT Voice Activity (VAD) software which detects the presence or absence of speech signals, and the ADT Comfort Noise Generator (CNG) which generates comfort noise at a specified level. These two algorithms make up ADT's VAD/CNG software package.

These algorithms are multi-thread re-entrant and can support an unlimited number of voice streams.

# 2. Software API

## 2.1. Channel Instantiation

```
VADCNG_Instance_t VADChan;
```

Allocate one instance per voice stream.

## 2.2. VADCNG_ADT_init( )

**Prototype:**

```
ADT_API void VADCNG_ADT_init(VADCNG_Instance_t *Inst,
                  ADT_Int16 Thres,
                  ADT_Int16 HangMSec,
                  ADT_Int16 VadFrameSize,
                  ADT_Int16 CngFrameSize,
                  ADT_Int16 WindowMSec,
                  ADT_Int16 SamplingRate,
                  ADT_Int16 CNGFIRLen);
```

**Inputs:**

| Argument Name | Range | Typical Values | Description |
|---|---|---|---|
| Inst | N/A | N/A | Pointer to VAD/CNG instance structure |
| Thres | 0..-95 | -45 | Speech level threshold, in dBm |
| HangMSec | 0..30000 | 100 | Hang time in milliseconds. The amount extra time at the end of a speech burst that is reported as voice. |
| VadFrameSize | 8..16000 | 80,160,240 | Number of samples input to VADCNG_ADT_vad |
| CngFrameSize | 8..16000 | 80,160,240 | Number of samples output from VADCNG_ADT_cng |
| WindowMSec | 2..16 | 5 | VAD analysis window size in milliseconds |
| SamplingRate | 8000, 16000 | | Sampling rate, in Hz |
| CNGFIRLen | 0..32 | 0 | Noise generation FIR filter size.<br>0 - Use random table lookup.<br>1..32 - FIR filter size. |

**Outputs:**

None

**Returns:**

None

## 2.3. VADCNG_ADT_vad( )

**Prototype:**

```
ADT_API ADT_Int16 VADCNG_ADT_vad (VADCNG_Instance_t *Inst,
                                  ADT_Int16 pcm_in[],
                                  ADT_Int16 *VadLvl);
```

**Inputs:**

Inst – pointer to VAD/CNG channel structure
pcm_in – pointer to 16-bit linear input PCM buffer

**Outputs:**

*VadLvl = Current noise level, in dBm

**Returns:**

VOICE, NOISE, or SILENCE: vad flag types are defined in the header file.

## 2.4. VADCNG_ADT_cng( )

**Prototype:**

```
ADT_API ADT_Int16 VADCNG_ADT_cng (VADCNG_Instance_t *Inst,
                                  ADT_Int16 VadFlag,
                                  ADT_Int16 Level,
                                  ADT_Int16 OutputBuffer[]);
```

**Inputs:**

Inst – pointer to VAD/CNG channel structure
VadFlag – VOICE: Leave output buffer unchanged.
Non VOICE: Generate noise for NOISE or SILENCE
Level – noise level to be generated, in dBm

**Outputs:**

OutputBuffer – pointer to PCM output buffer (16-bit PCM)

**Returns:**

0 – Success
1 - Failure

## 2.5. VAD_ADT_getStatus( )

**Prototype:**

```
ADT_API void VAD_ADT_getStatus (VADCNG_Instance_t* Inst,
                                VADCNG_Status_t* Status);
```

**Inputs:**

Inst – pointer to VAD/CNG channel structure

**Outputs:**

Status – pointer to VAD_Status_t structure to receive status data.

| Status Fields | Description |
|---|---|
| VadFrameSize | Number of samples that will be processed with each call to VADCNG_ADT_vad. |
| CngFrameSize | Number of samples that will be processed with each call to VADCNG_ADT_cng. |
| SortEnegry | Current energy level of voice stream |
| LastVad | Current state (VOICE, NOISE, or SILENCE) of voice stream. |

**Returns:**

None.

## 2.6. VAD_ADT_getInstanceSize( )

**Prototype:**

```
ADT_API ADT_UInt16 VAD_ADT_getInstanceSize (void);
```

**Inputs:**

None

**Outputs:**

None

**Returns:**

Size in bytes of the VADCNG_Instance_t instance structure. May be used to dynamically allocate instance structures.

## 2.7. VAD_ADT_LibVersion( )

**Prototype:**

```
ADT_API ADT_UInt16 VAD_ADT_getLibVersion (void);
```

**Inputs:**
None

**Outputs:**
None

**Returns:**
VAD library version number. May be compared against the #define variable 'VAD_LIB_VER' to verify that a build's library and header files match.

## 2.8. VAD_ADT_APIVersion( )

**Prototype:**

```
ADT_API ADT_UInt16 VAD_ADT_getAPIVersion (void);
```

**Inputs:**
None

**Outputs:**
None

**Returns:**
VAD API version number. May be compared against the #define variable 'VAD_API_VER' to verify that a build's library and header files match.

# 3. Sample Host Software

```
#include "vadcng_user.h"

#define MAX_FRAME_SIZE 320

ADT_Int16  dBmThreshold = -45;
ADT_Int16  hangTimeMsec = 100;
ADT_Int16  vadFrameSamps = 80;
ADT_Int16  cngFrameSamps = 80;
ADT_Int16  windowMsec = 5;
ADT_Int16  SamplingRate = 8000;
ADT_Int16  cngFIRLength = 0;
ADT_Int16  InSamples[MAX_FRAME_SIZE], OutSamples[MAX_FRAME_SIZE];
ADT_Int16  VADLevel;
ADT_Int16  VADStatus;

VADCNG_Instance_t VADChan;
```

```
VADCNG_ADT_init (&VADChan,
                  dBmThreshold,
                   hangTimeMsec,
                  vadFrameSamps,
                  cngFrameSamps,
                   windowMsec,
                  SamplingRate,
                  cngFIRLength);

VADStatus = VADCNG_ADT_vad (&VADChan, InSamples, &VADLevel);

...

VADCNG_ADT_cng (&VADChan, VADStatus, VADLevel, OutSamples);
```